CS & IT ENGINEERING

Programming in C

Miscellaneous Topics

Chapter 07

Lec- 0 01

By- Pankaj Sharma sir

TOPICS TO BE COVERED

Miscellaneous Topics

1. Consider the program in a hypothetical lang. that allow global variable and a choice of static & dynamic scoping.

20

```
int i ;
Program main(){
        i = 10;
        call f();
        }
Procedure f(){
        int i = 20
        call g();
        }
Procedure g(){
        print(i);
        }
```

let x : value under static scoping
y : value under dyn. scoping

1. Consider the program in a hypothetical lang. that allow global variable and a choice of static & dynamic scoping.

```
int i ;
Program main(){
        i = 10;
        call f();
        }
Procedure f(){
        int i = 20;
        call g();
        }
Procedure g(){
        print(i);
        }
```

let x : value under static scoping
    y : value under dyn. scoping

main → f() → g()

i_{global}: $\boxed{\cancel{0}\ 10}$

i_f: $\boxed{20}$

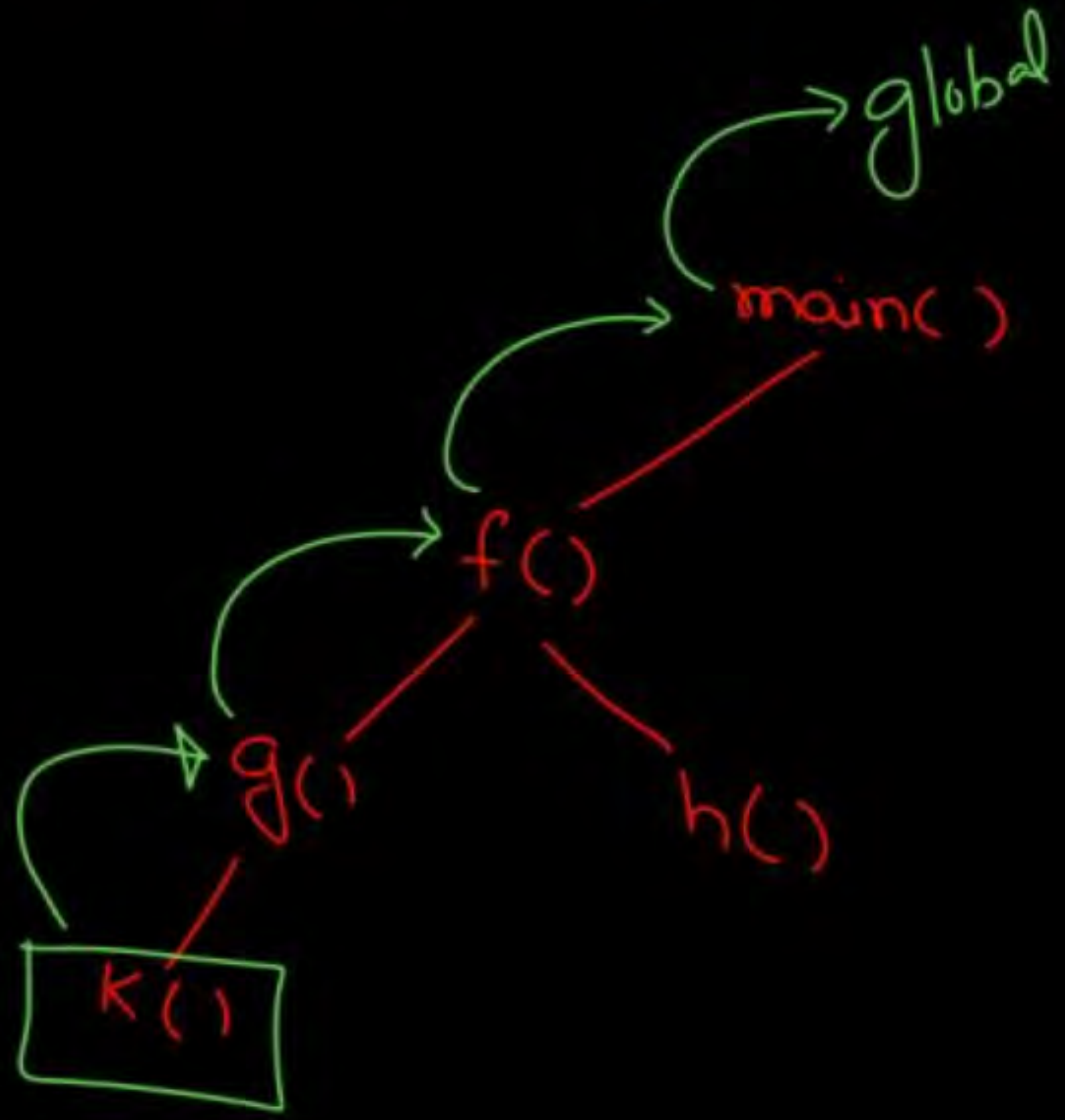
$i_{global} \leftarrow$ print(i);

$x = 10$

scoping

static scoping | Dynamic scoping

→ decided at compile time → Run time



→ calling function

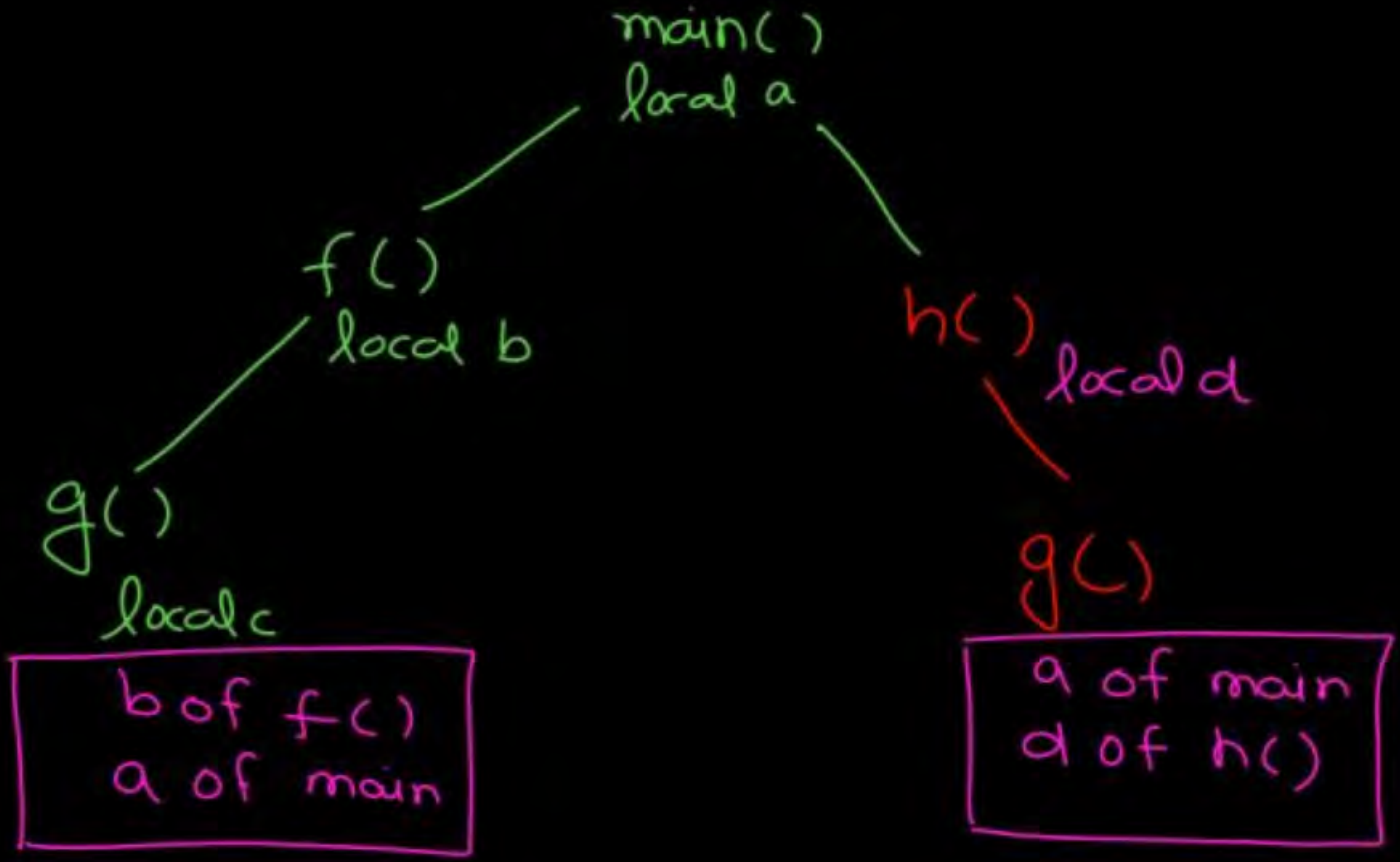
```
void main(){
    =
    printf("%d",a);
    =
}
```

main()
local c

f()
local b
c of main

g()
local a
c of main
b of f()

```
main(){
    int c;
    —
    f()
    =
}
f() {
    int b;
    =  print(c);
    g();
    =
}
g() {
    int a;
    =
}
```




```
void main(){
    int a;

    f();
    h()

f(){
    int b

    g();

}
g(){ int c

}
```

Comma Operator

1) Comma works as a seperator.

```
int x = 10, y = 20, z = 30;
```

OR

```
int x = 10;
int y = 20;
int z = 30;
```

2) Comma works as an operator.

$$x = (3, 4, 5); \qquad \boxed{x = 5}$$

$$Var = (Exp1, Exp2, Exp3, \ldots \ldots Exp_n)$$

All these exp. are evaluated from left to right and the final value is the right most exp. value.

First exp1 is eval. and simply discarded int i;

$$i = \left(\overset{6}{\cancel{\texttt{printf("Pankaj")}}}\ ,\ \overset{16}{\cancel{10+6}}\right)$$

printf("%d", i);

O/P :

Pankaj16

```c
int i=1,j;

j = (i=i+2, i++, i+9);

printf("%d %d", i,j);
```

13

4+9

4 13

i ~~1~~ ~~3~~ 4

j

3. Comma $\Rightarrow$ least priority operator

$a = 3, 4, 5;$

$\Downarrow$

1

$(a = 3), 4, 5;$

= is having more priority than ,

a

$\boxed{3}$

$3, 4, 5;$

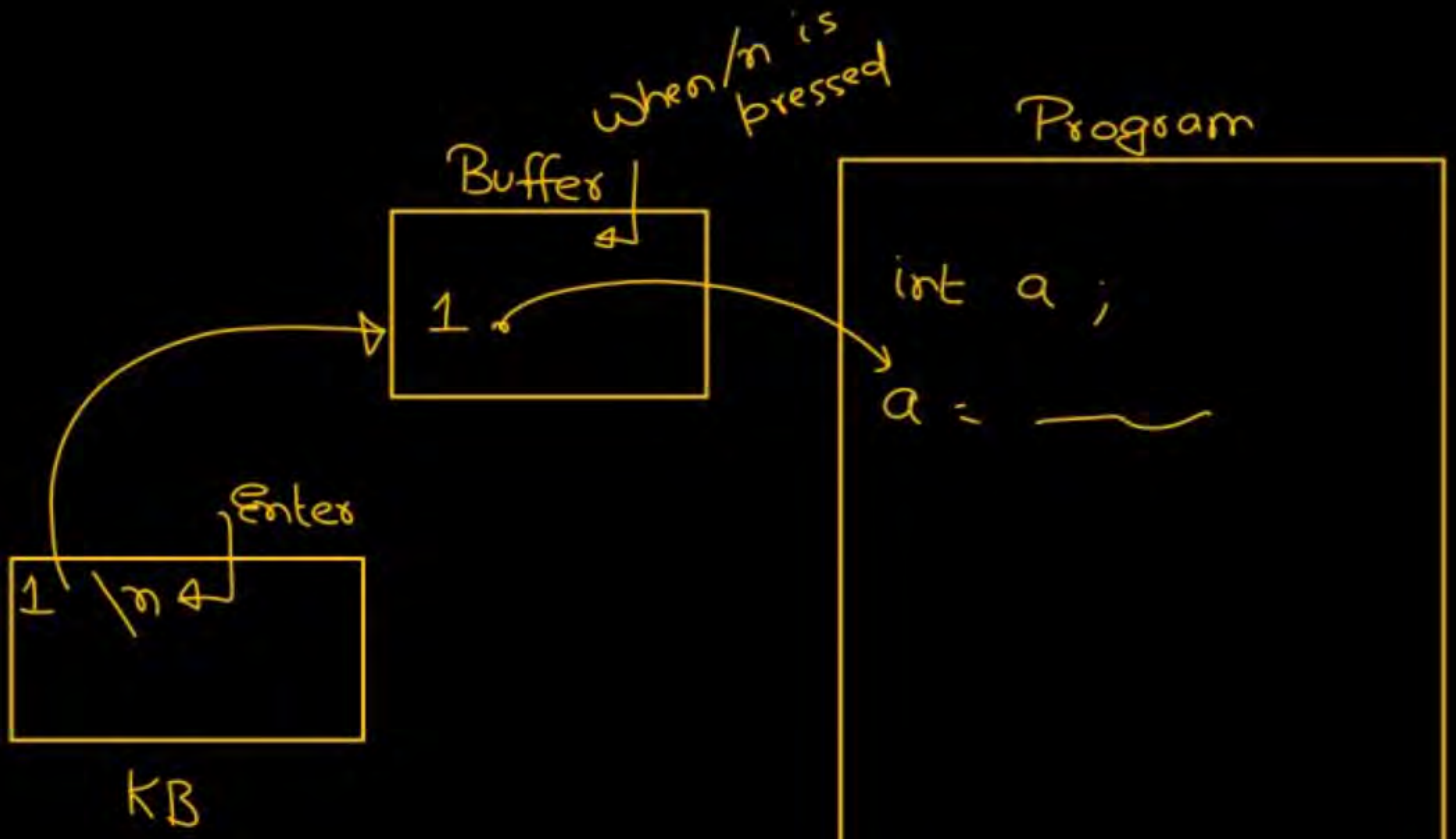
3;

4; OK

5;

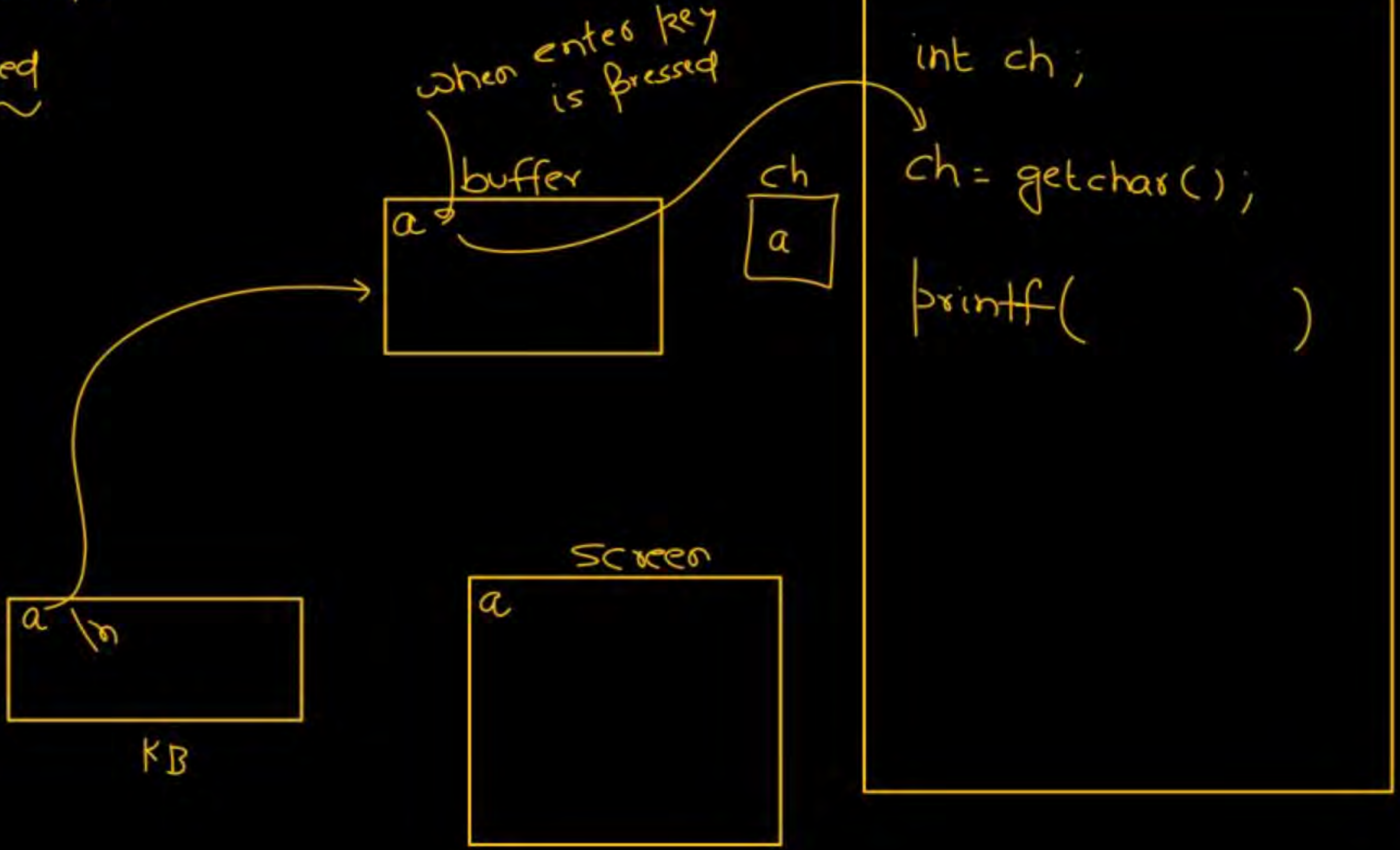
getchar() / getch / getche

1. Buffered or not ?
2. echoed or not ?



getchar()

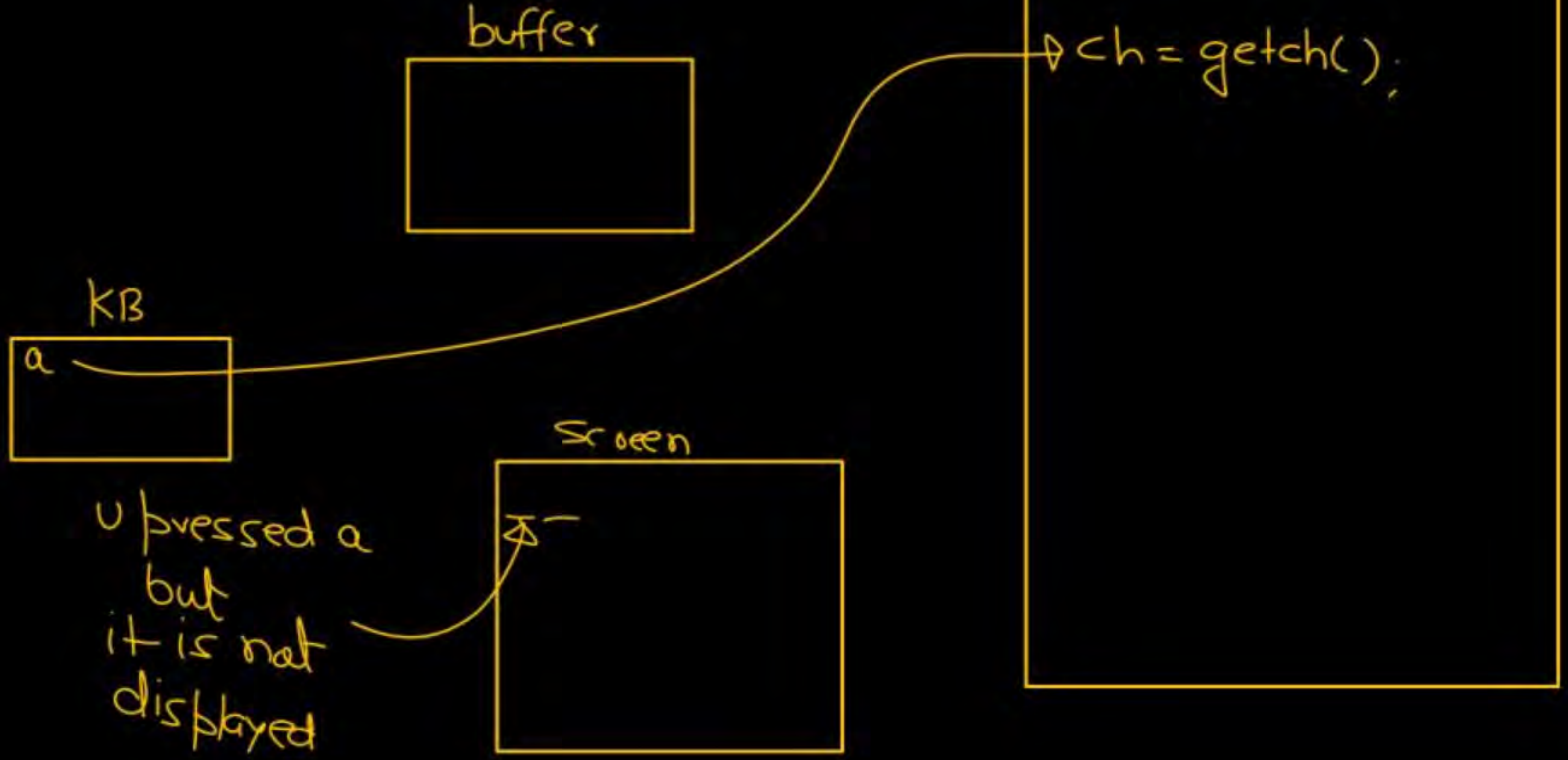
1. Buffered ✓
2. Echoed



getch()

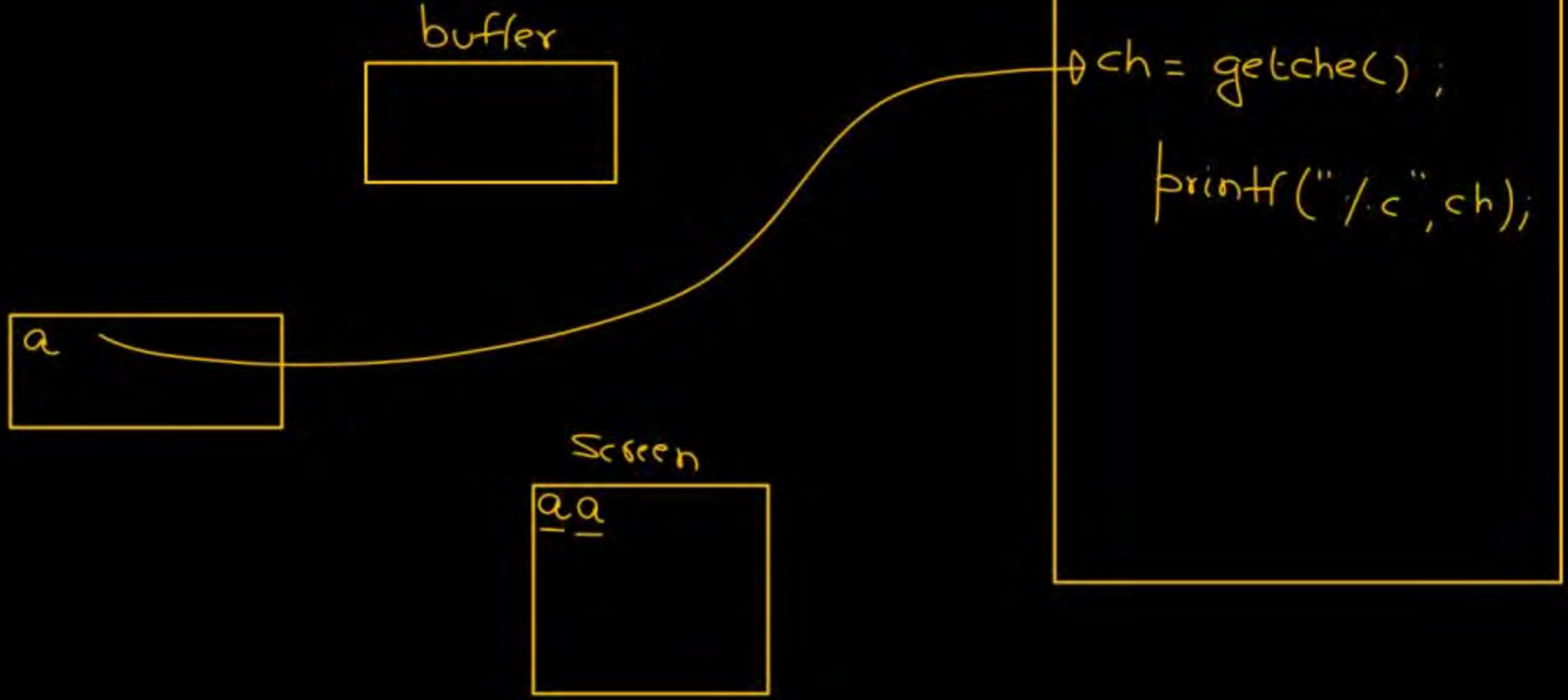
① Unbuffered

② Unechoed ✓



getche()

1. Unbuffered
2. Echo



Pre-processor directives

(i) Macro

(ii) File inclusion

```c
#define MAX 20
#include <stdio.h>
void main(){

    printf("%d", MAX);

}
```

.c

$\xrightarrow{\text{Pre processor}}$

```c
// content of stdio.h
void main(){

    printf("%d",20);

}
```

.i

```c
#define square(x) x*x
void main()
{
    int i;
    i = square(5+3);
    printf("%d", i);
}
```

(23)

$\Longrightarrow i = 5 + 3 \times 5 + 3$

$5 + 15 + 3$

23

```c
#define square(x) (x)*(x)
```

Union

✓ All member share common memory space.

```
struct P{
    int a;     → 4
    char ch;   → 1
};

void main(){
    struct P u;   → 5
```

```
union Q {
    int a;     → 4
    char ch;   → 1
};

void main(){
    union Q v;
    sizeof(v) ⟹ max(4,1)
              = 4
```

char int float
1 4 8

```
union Q {
    char a;
    int b;
    float c;
};
void main(){
    union Q v;
    sizeof(v)
}
```

$\text{sizeof}(v) \rightarrow \max(1,4,8) = 8$

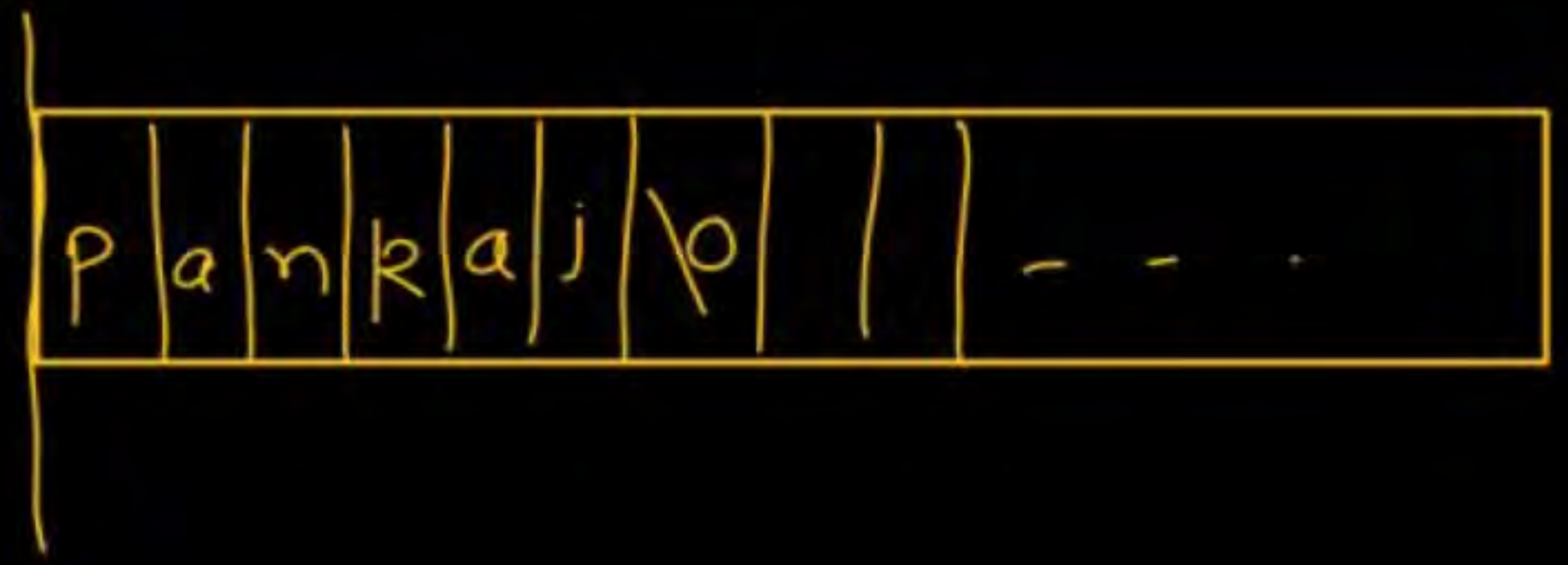
structure topic ⇒

bit field ✓

Padding ✓

```c
char name[20];
printf("Enter ur name");
scanf("%s", name);

printf("%s", name);
```



i/p : Pankaj

o/p : Pankaj

[\n]

```
char name[20];
printf("Enter ur name");
scanf("%s", name);
```

p	a	n	k	a	j	\0	

```
printf("%s", name);
```

```
gets(name)
```

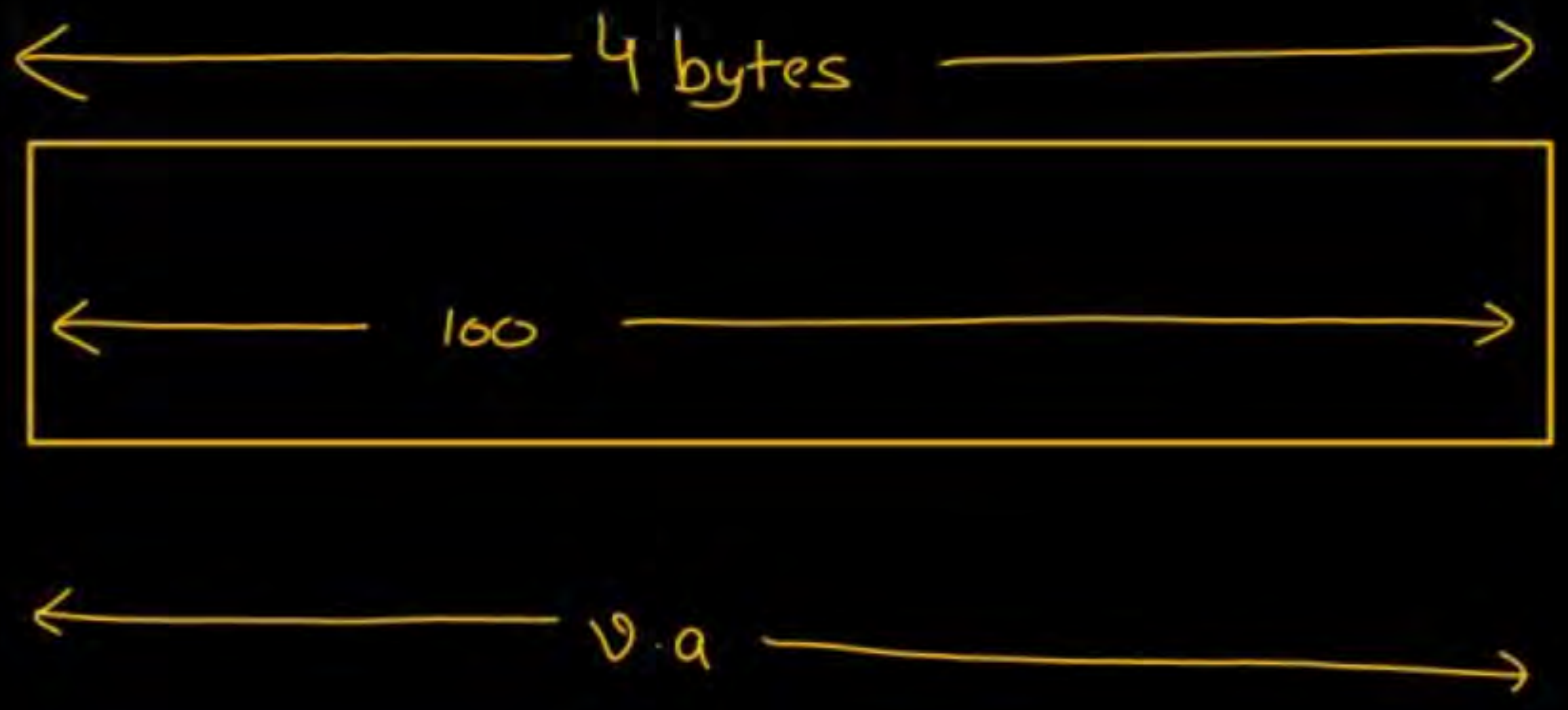
string

i/p : Pankaj Sharma

o/p : Pankaj

```
union Q {
        char c ;
        int a ;
        } v ;

void main() {
        Union Q v ;
        v.a = 100 ;
```



```
union Q {
        int a;
        char b;
        float c;
        }v;
void main(){
        v.c = 12.48;
        printf("%d", v.a);
```


THANK YOU GW
SOLDIERS !